PICK-DROP

Project documentation for testing

Admin panel, driver app, parent app, and self app. What the system does, how the accounts work, and what to test.

Document date 8 October 2026

Audience QA and testing team

API collection postman/Pick-Drop-Mobile-APIs.postman_collection.json

Payment notes postman/PAYMENT-API-CHANGES.md

1. What this product is

Pick-Drop is a school and commute transport system. A parent books a daily pick-up and drop for a child. A self user books the same kind of ride for themselves. A driver accepts the booking and runs the daily trip. An admin runs the web panel: approvals, assignment, payments, complaints, and emergencies.

There are four surfaces.

Surface	Who	How to test
Admin web panel	Admin staff	Browser
Driver app	Driver	Mobile app, or Postman against `/api/driver`
Parent app	Parent of a student	Mobile app, or Postman against `/api/parent`
Self app	Person riding for themselves	Mobile app, or Postman against `/api/self`

Parent and Self use the same API shape. Only the prefix changes. A parent token must not work on `/api/self`, and a self token must not work on `/api/parent`. A driver token must not work on either.

2. Access

Ask the project owner for the live URL, the admin login, and the test accounts before you start. Do not invent production passwords.

Typical local admin URL when the project runs under XAMPP:

```
http://localhost/Pick-Drop/public
```

API base:

```
http://localhost/Pick-Drop/public/api
```

Health check, no login required:

```
GET /api/ping  →  { "status": "ok" }
```

Every protected API call needs these headers:

```
Authorization: Bearer <token>
Accept: application/json
```

File uploads (KYC, vehicle photos, payment screenshot) use `multipart/form-data`. Image files must be 5 MB or smaller.

Login returns a token. Use that token until logout. After logout the same token must be rejected.

3. Accounts and the order they must follow

A new user cannot book or drive until an admin approves them. Test this order. Skipping a step should fail with a clear error.

Driver

1. `POST /api/driver/register` — name, phone, email, password. Referral code is optional.
2. `POST /api/driver/verify-email` — OTP. Until email is verified, later steps stay blocked.
3. `POST /api/driver/verification` — driver KYC. Required: date of birth, address, city, at least one service area, CNIC front, CNIC back, selfie, licence number, licence front, licence back, licence expiry (a future date), and terms accepted.
4. Admin opens **People** → **Driver KYC**, reviews the documents, and approves or rejects.
5. `POST /api/driver/vehicle-verification` — vehicle documents. Exact fields are in section 7.
6. Admin opens **People** → **Vehicle Verification** and approves or rejects.
7. The driver can go on duty with `PUT /api/driver/me/duty` and then see available requests.

Expected blocks:

- Vehicle form before email verification → 403.
- Vehicle form before KYC approval → 403. The message says to complete driver KYC first.
- A second submit while the current one is pending → 422.
- A submit after approval → 422.

Parent

1. `POST /api/parent/register`
2. Verify email.
3. `POST /api/parent/verification` — date of birth, address, city, CNIC front, CNIC back, selfie, terms accepted.
4. Admin approves under **People** → **Parent / Self KYC**.
5. Add a student: `POST /api/parent/students`.
6. Create a pick-up request. `student_id` is required. `type` must be `parent`.

Self

Same as parent, on `/api/self`, with these differences:

- There are no students. The rider is the account itself.
- A commute profile exists: `GET` and `POST /api/self/commute-profile`.
- Request `type` must be `self`.

Forgot password

These three calls are separate: `POST /api/driver/forgot-password`, `/api/parent/forgot-password`, and `/api/self/forgot-password`.

- A driver email on the parent or self forgot-password API must not receive a reset mail.
- A parent email on the driver forgot-password API must not receive a reset mail.
- An unknown email must not reveal that some other role owns that address.

4. Admin panel map

Log in on the web panel. The left menu is the test map.

Menu	What to check
Dashboard	Counts load. The page is not blank and shows no error.
Reports	Filters work. Export downloads a file.
People → Users	Create, edit, and disable a user.
People → Login logs	A login appears in this list.
People → Driver KYC	Pending list, open documents, approve, reject.
People → Parent / Self KYC	Same checks for parent and self.
People → Vehicle Verification	Photos open. Approve and reject. Owner-document photos are not a separate required pair.
Locations → Cities / Areas / Points	Add, edit, delete. A city used by a live request must not disappear without a clear error.
Vehicles → List / Categories	A category saved here appears in the driver vehicle form.
Operations → Pickup Requests	A new app request shows here. Assign a driver. Mark the driver payout.
Operations → Live fleet	The driver dot moves after the app sends a location.
Operations → Routes	Create a route and its stops.
Operations → Institutions	Schools listed here are what the parent app loads from `GET /api/parent/schools`.
Complaints	A driver or parent issue appears. Status can be changed.
SOS Alerts	A driver or parent SOS appears. Acknowledge, resolve, and End trip.
Calendar	A holiday blocks or skips that date for rides.
Payments	Invoice, screenshot receipt, mark received, reject, refund, print.
Pick-Drop Charges	The charge used when a request is priced.
Driver Payments	A payroll request from the driver app shows here.
Settings	Platform settings save and stay after refresh.

5. Main journey — test this first

Run this once with a parent and once with a self user. This path must work before anything else is signed off.

1. Admin has at least one city, one area, one vehicle category, and one charge.
2. Parent or self registers, verifies email, and submits KYC.
3. Admin approves KYC.
4. Parent adds a student. Self skips this step.

5. Parent or self creates a request with both a pick-up point and a drop point, times, and weekdays.
6. The request appears in the app list and in **Admin → Pickup Requests**.
7. An approved driver who is on duty sees it in `GET /api/driver/requests/available`.
8. The driver accepts it. The parent sees the driver on `GET /api/parent/requests/{id}/driver`.
9. On the ride day, the driver opens `GET /api/driver/rides/today`.
10. The driver starts the ride: `POST /api/driver/rides/{pickup_request_id}/start`.
11. The driver sends location: `POST /api/driver/location/update`. Parent tracking (`GET .../tracking`) shows the new point.
12. The driver marks pick-up, then drop.
13. `GET /api/driver/rides/today` shows `trip.can_end = true`.
14. The driver ends the trip: `POST /api/driver/rides/end`.
15. The parent rates the driver. The driver can rate the request.
16. Admin creates or sends the invoice. The parent pays by screenshot. Admin marks the receipt received. The invoice becomes paid.

> If step 13 stays false, do not force the end call. Read `trip.end_blocked_reason` and `trip.pending_passengers`. Those names are the passengers who are still not dropped and not marked as not going.

6. Booking rules

Create a booking with `POST /api/parent/requests` or `POST /api/self/requests`.

`service_type` is one of:

Value	Meaning
`both`	Morning pick-up and afternoon drop. This is the default.
`pickup_only`	Pick-up only.
`drop_only`	Drop only.

`trip_mode` is `round_trip` or `one_way`.

For a normal booking (no `stops` array):

- `type` is required and must match the app (`parent` or `self`).
- `city_id` is required.
- Pick-up point, pick-up latitude and longitude, drop point, and drop latitude and longitude are required.
- Both locations are required even when the service is pick-up only or drop only.
- `days` is a non-empty list of weekdays.
- Pick-up time (`H:i`, example `07:30`) is required unless the service is drop only.
- Drop time is required unless the service is pick-up only.
- Parent requests need a `student_id` that belongs to that parent.

An optional multi-stop body uses `stops[]` with `type` (`pickup` or `drop`), `point`, `lat`, and `lng`. Maximum 20 stops.

After create, also test:

- List the request and open it.
- Edit it before a driver is locked in.
- Cancel it. Call cancellation preview first: `GET .../cancellation-preview`.
- Renew, turn auto-renew on and off, and decline renewal.
- Skip a day (`POST .../attendance/skip`) and undo it.
- A skipped day must not appear as a normal ride for the driver.

7. Vehicle verification

`GET /api/driver/vehicle-verification` returns `vehicle_categories` for active categories: id, name, and passenger capacity. The app must build the category dropdown from this list.

`POST /api/driver/vehicle-verification` fields:

Field	Rule
`vehicle_name`	Required
`license_plate`	Required, unique
`registration_card_front`	Required image
`registration_card_back`	Required image
`vehicle_front_photo`	Required image
`vehicle_back_photo`	Required image
`number_plate_photo`	Required image
`owner_name`	Required
`vehicle_category_id`	Optional. Must be a real category id.
`vehicle_model`	Optional
`vehicle_color`	Optional
`owner_cnic_number`	Optional

> These two fields are removed. The app does not need to send them. Leaving them out must succeed: `owner_document_front` and `owner_document_back`.

ID	Action	Expected
VV-1	Submit with registration card, vehicle photos, number plate, and owner name. No owner-document photos.	201. Status is pending.
VV-2	Submit with no registration card front.	422. That field is required.
VV-3	Open the admin vehicle verification detail.	Registration and vehicle photos show. Missing owner-document photos do not block approval.
VV-4	Approve.	Driver next step moves forward and onboarding can complete.
VV-5	Reject.	The driver can submit again.
VV-6	Use a number plate that another driver already saved.	422.

8. Daily trip, end trip, not going, and emergency

These are the newest driver rules. When you can, test them on a day that has at least two accepted passengers.

Today screen

`GET /api/driver/rides/today`

Each passenger includes:

- `passenger_status`: `waiting`, `picked`, `dropped`, or `not_going`
- `not_going_reason`
- `can_mark_not_going` — true only after the trip has started, and only if that passenger is not already picked and not already marked not going

The trip object includes:

- `can_end`
- `end_blocked_reason`
- `emergency_open`
- `pending_passengers` — names still not settled
- Counts: total, picked, dropped, not going

The app should show the End Trip button only when `can_end` is true.

Start and finish a passenger

Call	When
`POST /api/driver/rides/{id}/start`	Start that passenger's day
`POST /api/driver/rides/{id}/arrive`	Driver reached the point
`POST /api/driver/rides/{ride}/pickup`	Passenger is in the vehicle
`POST /api/driver/rides/{ride}/drop`	Passenger is dropped

End trip

`POST /api/driver/rides/end` with an empty body.

ID	Setup	Expected
TR-1	Trip not started	422. Start the trip first. `can_end` is false.
TR-2	Started, one passenger still waiting	422. The message names that passenger. `pending_passengers` lists them.
TR-3	Every passenger dropped	200. `can_end` becomes false and `ended` is true. A second end call fails.
TR-4	One passenger dropped, one marked not going	200. Not-going does not block the end.
TR-5	No passengers today	422.

Passenger does not want to go

`POST /api/driver/rides/{pickup_request_id}/not-going`

```
{ "reason": "Does not want to go today" }
```

`reason` is required. Maximum 500 characters.

ID	Setup	Expected
NG-1	Trip not started	422
NG-2	Trip started, passenger not picked yet	200. Status `not_going`. Reason is saved. The parent gets a notification. The admin panel gets a warning.
NG-3	Passenger already picked up	422. The driver must drop them instead.
NG-4	Empty reason	422
NG-5	After NG-2, end the trip when everyone else is dropped	End succeeds.

Emergency

`POST /api/driver/sos`

Field	Rule
`type`	Optional. One of `accident`, `medical`, `breakdown`, `security`.
`message`	Optional, maximum 500 characters.
`lat`, `lng`	Optional. If omitted, the last driver location is used.
`pickup_request_id`	Optional. It must belong to this driver.

ID	Action	Expected
EM-1	Send SOS with `type=accident` during an active trip	200. The alert is on **Admin** → **SOS Alerts**. The type is visible.
EM-2	Driver calls `POST /api/driver/rides/end` while that SOS is still open or acknowledged	422. `can_end` is false. The reason says admin will end this trip.
EM-3	Admin opens the SOS and clicks **End trip**	The trip ends. The SOS becomes resolved. The driver can no longer end it themselves.
EM-4	Admin only acknowledges and does not end the trip	The driver still cannot end the trip.
EM-5	A resolved SOS from an earlier day	It must not keep blocking a new day. An open or acknowledged SOS from today still locks the driver.
EM-6	SOS `type` outside the four names, for example `fire`	422
EM-7	Parent SOS `POST /api/parent/sos`	It appears in the admin SOS list. It does not use the driver end-trip lock.

9. Payments

Field-level notes are also in `postman/PAYMENT-API-CHANGES.md`. The same calls exist for parent and self. Examples below use the parent prefix.

1. `GET /api/parent/payment-methods` — bank, JazzCash, and EasyPaisa details the app should show.
2. `GET /api/parent/invoices` and `GET /api/parent/invoices/{id}`.
3. The user pays outside the app, then calls `POST /api/parent/invoices/{id}/pay/screenshot`.
   - `proof` image is required.
   - `method` is optional: `bank_transfer`, `jazzcash`, `easypaisa`, or `manual`.
   - `reference` and `notes` are optional.
4. The response status is `not_received`. The invoice is not paid yet.
5. Admin opens **Payments**, reviews the screenshot, and marks it received.
6. The app calls `GET /api/parent/invoices/{id}/pay/status`. `receipt_status` is `received` and `paid` is true.

ID	Action	Expected
PY-1	Screenshot on an already paid invoice	422
PY-2	A second screenshot while the first is still `not_received`	The new file replaces the first screenshot.
PY-3	Admin rejects the receipt	The app status is not paid. The user can send a new screenshot.
PY-4	`POST .../pay/bank` without `proof`	422. Proof is required. Reference is optional.
PY-5	Old direct JazzCash or EasyPaisa pay URLs	Those URLs are removed. Use the screenshot call with `method=jazzcash` or `method=easypaisa`.

10. Other driver checks

Area	Call	Expected
Duty	`PUT /api/driver/me/duty`	An off-duty driver does not receive new available requests.
Available / accepted	`GET /api/driver/requests/available` and `/accepted`	An accepted request leaves the available list.
Accept / reject	`POST .../accept` and `.../reject`	A second driver cannot accept the same request.
Cover	`POST .../unavailable`, then another driver uses `GET /api/driver/requests/cover` and `POST /api/driver/cover/{id}/accept`	The original driver is replaced for that cover. The parent sees the replacement.
Live status	`POST /api/driver/status/update`	The parent today-status changes.
Issue	`POST /api/driver/issues`	It shows under Admin → Complaints and on the parent issues list.
Chat	Messages on a request, both sides	Only the assigned pair can read that thread.
Earnings / payroll	`GET /api/driver/earnings`, `POST /api/driver/payroll/request`	The request shows under Admin → Driver Payments.
Notifications	List, mark one read, mark all read	The unread count drops.
Change password	Wrong current password	422. The password stays unchanged.
Delete account	`DELETE /api/driver/account`	The token stops working.

11. Other parent and self checks

Area	What to confirm
Profile	Name and phone update. `GET /me` returns the new values.
Students	A parent can add, edit, and delete. Self has no students API.
Tracking	The map point matches the last driver location call.
OTP	`GET .../otp` returns today's code only for that user's request.
Contact	The phone is shown only after a driver is assigned.
Ratings	One rating per completed request. Record whether a second rating fails or updates.
Holidays	A date on the admin calendar is returned by `GET /api/parent/holidays`.
Wallet	The balance on `GET /api/parent/wallet` matches admin after a payment.
Notification preferences	Turning a type off stops that push. If it still arrives, note that in the bug.
Issues	A parent complaint appears in Admin → Complaints.

12. API list

Replace `{id}` with the real id from the previous response. Every call except register, login, forgot password, reset password, verify email, resend OTP, cities, and `/api/ping` needs a Bearer token.

Driver — /api/driver

Method	Path
POST	`/register` `/login` `/forgot-password` `/reset-password` `/verify-email` `/resend/otp` `/logout`
GET	`/cities` `/cities/{city}/areas`
GET, POST	`/verification`
GET, POST	`/vehicle-verification`
GET, PUT	`/me`
PUT	`/me/duty` `/account/change-password` `/account/payment-details`
GET	`/account/payment-details`
DELETE	`/account`
GET, POST	`/service-areas`
GET	`/requests/available` `/requests/accepted` `/requests/cover`
POST	`/requests/{id}/accept` `/requests/{id}/reject` `/requests/{id}/status` `/requests/{id}/unavailable`
GET, POST	`/requests/{id}/messages`
GET	`/requests/{id}/contact`
POST	`/requests/{id}/ratings`
GET	`/rides/today` `/rides`
POST	`/rides/end` `/rides/{id}/start` `/rides/{id}/arrive` `/rides/{id}/not-going` `/rides/{id}/pickup` `/rides/{id}/drop`
POST	`/location/update` `/status/update` `/issues` `/sos`
GET	`/issues/today` `/notifications` `/holidays` `/earnings` `/payroll` `/payroll/{id}` `/payrolls` `/payrolls/{id}`
POST	`/notifications/mark-all-read` `/notifications/{id}/read` `/phone/verify/send` `/phone/verify`
POST	`/payroll/request` `/payroll/preview` `/cover/{id}/accept` `/device-token`

Parent — /api/parent and Self — /api/self

The same paths exist on both prefixes, except students and the commute profile.

Method	Path	Who
POST	`/register` `/login` `/forgot-password` `/reset-password` `/logout`	Both
POST	`/email/verify/send` `/email/verify` `/phone/verify/send` `/phone/verify`	Both
GET, POST	`/verification`	Both
GET, POST	`/commute-profile`	Self only
GET, POST, PUT, DELETE	`/students` and `/students/{id}`	Parent only
GET, PUT	`/me`	Both
PUT	`/account/change-password`	Both
DELETE	`/account`	Both
GET	`/cities` `/cities/{city}/areas` `/cities/{city}/points` `/schools` `/wallet` `/holidays` `/payment-methods`	Both
GET, POST	`/requests`	Both
GET, PUT, DELETE	`/requests/{id}`	Both
GET	`/requests/{id}/driver` `/tracking` `/replacements` `/attendance` `/ratings` `/contact` `/cancellation-preview` `/otp`	Both
POST	`/requests/{id}/renew` `/auto-renew` `/decline-renewal` `/attendance/skip` `/ratings`	Both
DELETE	`/requests/{id}/attendance/skip`	Both
GET	`/invoices` `/invoices/{id}` `/invoices/{id}/pay/status`	Both
POST	`/invoices/{id}/pay/stripe` `/pay/bank` `/pay/screenshot`	Both
GET	`/trips/recent` `/trips/today-status` `/issues` `/notifications` `/notification-preferences` `/sos`	Both
POST	`/issues` `/sos` `/device-token` `/notifications/mark-all-read` `/notifications/{id}/read`	Both
GET, PUT	`/notification-preferences`	Both
GET, POST	`/requests/{id}/messages`	Both

13. Cross-checks

- A parent token on `/api/driver/...` returns 401 or 403.
- A driver token on `/api/parent/...` returns 401 or 403.
- A parent token on `/api/self/...` returns an account-type error.
- Request `type=self` from the parent app is rejected.
- Two drivers accepting the same request: only the first succeeds.
- Ending a trip while one child is still in the vehicle is rejected.

- An SOS during a trip blocks the driver end button until an admin ends the trip.
- Owner document front and back are not required.
- A payment screenshot does not mark the invoice paid until an admin confirms it.
- A holiday date does not generate a normal ride.
- After logout, the old token fails.

14. How to report a bug

Include all of these:

1. Surface: Admin, Driver app, Parent app, or Self app.
2. Account email and role used.
3. Numbered steps.
4. Request method and path, if it is an API bug.
5. Response status code and the `message` field.
6. Expected result, taken from this document.
7. Actual result.
8. Screenshot or the response body.
9. Time of the test.

Level	Meaning
Blocker	Register, login, booking, accept, pay, or trip end cannot be completed.
Major	A rule in the vehicle, trip, or payment sections fails.
Minor	Wrong text, layout, or a field that is saved but displayed incorrectly.
Note	The message is unclear. The feature still works.

Pick-Drop testing document. 8 October 2026. Give the testers this PDF together with the Postman collection and the admin URL plus test accounts.